

10 Different Examples of While Loops in Python

A Guide for BCA Students

Example 1: Basic While Loop with Counter

Concept: Using a counter variable to control loop iterations

```
# Print numbers from 1 to 5
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output:

```
1
2
3
4
5
```

Explanation: The loop continues as long as `count` is less than or equal to 5. After each iteration, `count` is incremented by 1. Once `count` becomes 6, the condition becomes false and the loop terminates.

Example 2: While Loop with User Input

Concept: Repeating a loop until the user provides specific input

```
# Keep asking for password until correct
password = "python123"
attempt = ""

while attempt != password:
    attempt = input("Enter password: ")
    if attempt == password:
        print("Access granted!")
    else:
        print("Incorrect password. Try again.")
```

Output:

```
Enter password: hello
Incorrect password. Try again.
Enter password: python123
```

Access granted!

Explanation: The loop continues until the user enters the correct password. This is useful for authentication and validation tasks.

Example 3: While Loop with String Manipulation

Concept: Processing strings character by character using a while loop with index

```
# Print each character with its position
message = "PYTHON"
index = 0

while index < len(message):
    print(f"Position {index}: {message[index]}")
    index += 1
```

Output:

```
Position 0: P
Position 1: Y
Position 2: T
Position 3: H
Position 4: O
Position 5: N
```

Explanation: Using `len()` function, we determine the string length and iterate through each character using an index variable.

Example 4: While Loop with Condition (Validation)

Concept: Validating user input within a specified range

```
# Accept a number between 1 and 10
number = 0

while number < 1 or number > 10:
    try:
        number = int(input("Enter a number between 1 and 10: "))
        if number < 1 or number > 10:
            print("Invalid input! Please try again.")
    except ValueError:
        print("Please enter a valid integer.")
        number = 0
```

```
print(f"You entered: {number}")
```

Output:

```
Enter a number between 1 and 10: 15
Invalid input! Please try again.
Enter a number between 1 and 10: abc
Please enter a valid integer.
Enter a number between 1 and 10: 7
You entered: 7
```

Explanation: This example demonstrates input validation, a common real-world requirement in applications.

Example 5: While Loop with List Processing

Concept: Iterating through a list using index and while loop

```
# Display shopping items from a list
shopping_list = ["Milk", "Bread", "Eggs", "Butter", "Cheese"]
item_number = 0

while item_number < len(shopping_list):
    print(f"{item_number + 1}. {shopping_list[item_number]}")
    item_number += 1
```

Output:

```
1. Milk
2. Bread
3. Eggs
4. Butter
5. Cheese
```

Explanation: While loops can be used with lists when you need more control over iteration than a for loop provides.

Example 6: While Loop with Break Statement

Concept: Exiting a loop prematurely when a condition is met

```
# Find the first number divisible by both 3 and 5
number = 1

while True:
    if number % 3 == 0 and number % 5 == 0:
```

```

        print(f"Found: {number}")
        break
    number += 1
    if number > 30: # Safety check to prevent infinite loop
        break

print("Search complete")

```

Output:

Found: 15
Search complete

Explanation: while True creates an infinite loop that must be exited using break. This is useful when the exit condition is complex or uncertain.

Example 7: While Loop with Continue Statement

Concept: Skipping certain iterations based on a condition

```

# Print odd numbers from 1 to 10, skip even numbers
num = 0

```

```

while num < 10:
    num += 1
    if num % 2 == 0: # Skip even numbers
        continue
    print(num)

```

Output:

1
3
5
7
9

Explanation: The continue statement skips the print() statement for even numbers and jumps to the next iteration.

Example 8: While Loop with Nested Conditions

Concept: Creating a menu-driven program using while loop

```

# Simple calculator menu
while True:
    print("\n--- Calculator Menu ---")

```

```

print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Exit")

choice = input("Enter your choice (1-4): ")

if choice == "4":
    print("Thank you for using the calculator!")
    break
elif choice in ["1", "2", "3"]:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    if choice == "1":
        print(f"Result: {num1 + num2}")
    elif choice == "2":
        print(f"Result: {num1 - num2}")
    elif choice == "3":
        print(f"Result: {num1 * num2}")
    else:
        print("Invalid choice! Please try again.")

```

Output:

```

--- Calculator Menu ---
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice (1-4): 1
Enter first number: 10
Enter second number: 5
Result: 15.0

```

```

--- Calculator Menu ---
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice (1-4): 4
Thank you for using the calculator!

```

Explanation: Menu-driven programs use while loops to keep running until the user chooses to exit.

Example 9: While Loop with Multiple Counters

Concept: Using multiple variables to control different aspects of the loop

```
# Print a pattern: decreasing pyramid of stars
rows = 5
current_row = rows

while current_row > 0:
    stars = current_row
    while stars > 0:
        print("*", end=" ")
        stars -= 1
    print() # New line after each row
    current_row -= 1
```

Output:

```
* * * * *
* * * *
* * *
* *
*
```

Explanation: Nested while loops with multiple counters create complex patterns. This is useful for generating ASCII art and visualizations.

Example 10: While Loop with Accumulator Pattern

Concept: Accumulating values in a variable within a loop

```
# Calculate sum of numbers until user enters 0
total = 0
count = 0

while True:
    number = int(input("Enter a number (0 to stop): "))
    if number == 0:
        break
    total += number
    count += 1

if count > 0:
    print(f"Sum: {total}")
    print(f"Count: {count}")
    print(f"Average: {total / count:.2f}")
```

```

else:
    print("No numbers entered.")

```

Output:

```

Enter a number (0 to stop): 10
Enter a number (0 to stop): 20
Enter a number (0 to stop): 30
Enter a number (0 to stop): 0
Sum: 60
Count: 3
Average: 20.00

```

Explanation: The accumulator pattern is fundamental in programming for calculating totals, averages, and counts. The `total` variable accumulates values with each iteration.

Summary Table

Example	Key Concept	Use Case
1	Basic counter	Simple counting loops
2	User input validation	Authentication, user interaction
3	String processing with index	Character-level string manipulation
4	Input validation	Ensuring data meets criteria
5	List processing	Working with lists using index
6	Break statement	Early loop termination
7	Continue statement	Skipping specific iterations
8	Nested conditions	Menu-driven applications
9	Multiple counters	Complex pattern generation
10	Accumulator pattern	Calculating sums, averages, totals

While Loop vs For Loop: When to Use Each

Feature	While Loop	For Loop
Known iterations	Not ideal	Better choice
Unknown iterations	Better choice	Not ideal
User input control	Better choice	Less suitable
List/sequence iteration	Works but verbose	More elegant
Complex conditions	Better choice	Awkward
Example	User password entry	Iterate through list items

Common Mistakes to Avoid

Infinite Loop - Forgetting to update loop variable

```
count = 1
while count <= 5:
    print(count)
    # count is never incremented - infinite loop!
```

Correct:

```
count = 1
while count <= 5:
    print(count)
    count += 1 # Update the loop variable
```

Wrong comparison operator

```
count = 1
while count < 5:
    print(count)
    count += 1
# Prints 1, 2, 3, 4 (not 5) - might be unintended
```

Correct (if you want 1 to 5):

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Logic error with break/continue

```
num = 0
while num < 10:
    if num == 5:
```

```

        break
    # num never increments if num != 5
    num += 1

```

Correct:

```

num = 0
while num < 10:
    num += 1 # Update before condition check
    if num == 5:
        break

```

Practice Challenges for BCA Students

1. **Number Guessing Game:** Create a while loop that picks a random number and lets the user guess until they get it right.
2. **Factorial Calculator:** Use a while loop to calculate the factorial of a number entered by the user.
3. **Password Strength Checker:** Create a while loop that keeps asking for a password until it meets certain criteria (length 8, contains numbers and letters).
4. **Prime Number Checker:** Use a while loop to check if a number is prime.
5. **Word Repetition:** Build a while loop that takes a word from the user and prints it an increasing number of times:

```

apple
apple apple
apple apple apple

```

Key Points to Remember

While loops are ideal when the number of iterations is **unknown**
 Always ensure the loop termination condition will eventually become **true**
 Use **break** to exit a loop early
 Use **continue** to skip to the next iteration
 Be careful of **infinite loops** - always test your code
 While loops give you more **control** than for loops but require careful management
